



Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencia de la Computación

Clase 24: Simulacro Interrogación: Perforce (solución)

Francisco Gazitúa Requena (fco_gazitua_requena@uc.cl)

IIC2113 Diseño Detallado de Software

7 de Octubre, 2025

Interrogación 2024-1

Interrogación 2024-1

Perforce es un sistema de control de versiones que permite gestionar el desarrollo de software mediante la organización y el control de cambios en el código fuente. Es parecido a Git pero centralizado.

Interrogación 2024-1

Perforce es un sistema de control de versiones que permite gestionar el desarrollo de software mediante la organización y el control de cambios en el código fuente. Es parecido a Git pero centralizado.

Cada usuario en Perforce puede tener varios clientes. Y cada cliente tiene un depot. El depot es una ruta a una carpeta dentro del servidor. El usuario podrá extraer y modificar todos los archivos dentro de su depot (incluyendo, crear carpetas).

Interrogación 2024-1

Perforce es un sistema de control de versiones que permite gestionar el desarrollo de software mediante la organización y el control de cambios en el código fuente. Es parecido a Git pero centralizado.

Cada usuario en Perforce puede tener varios clientes. Y cada cliente tiene un depot. El depot es una ruta a una carpeta dentro del servidor. El usuario podrá extraer y modificar todos los archivos dentro de su depot (incluyendo, crear carpetas).

En simple, existen usuarios. Cada usuario tiene varios clientes. Cada cliente tiene una carpeta “depot” en el servidor donde puede crear, modificar y borrar archivos.

Interrogación 2024-1

Perforce es un sistema de control de versiones que permite gestionar el desarrollo de software mediante la organización y el control de cambios en el código fuente. Es parecido a Git pero centralizado.

Cada usuario en Perforce puede tener varios clientes. Y cada cliente tiene un depot. El depot es una ruta a una carpeta dentro del servidor. El usuario podrá extraer y modificar todos los archivos dentro de su depot (incluyendo, crear carpetas).

En simple, existen usuarios. Cada usuario tiene varios clientes. Cada cliente tiene una carpeta “depot” en el servidor donde puede crear, modificar y borrar archivos.

Para crear/modificar/borrar archivos les damos una librería:

- `void Add(FilePath filePath, string? content).`
- `bool Exists(FilePath path).`
- `Files Get(FilePath path, string pathToFile).`

El proyecto `Perforce` es una primera versión de un servidor de *Perforce*. Su clase de entrada es `Controller.cs` y gestiona tanto a los clientes como a sus archivos.

El proyecto *Perforce* es una primera versión de un servidor de *Perforce*. Su clase de entrada es `Controller.cs` y gestiona tanto a los clientes como a sus archivos.

`Controller.cs` tiene un método de entrada que recibe un `Request` con información asociada a lo que el usuario quiere hacer. En particular,

- Si `request.Type` es “client”, se crea un cliente o se obtiene su información.
- Si `request.Type` es “sync”, se obtiene información de los archivo(s) en el depot.
- Si `request.Type` es “submit”, se suben cambios al servidor. Esta solicitud está relacionada con una lista de cambios, los cuales pueden ser agregar, editar o eliminar archivos. Se espera soportar más tipos de cambio en el futuro.

El proyecto *Perforce* es una primera versión de un servidor de *Perforce*. Su clase de entrada es `Controller.cs` y gestiona tanto a los clientes como a sus archivos.

`Controller.cs` tiene un método de entrada que recibe un `Request` con información asociada a lo que el usuario quiere hacer. En particular,

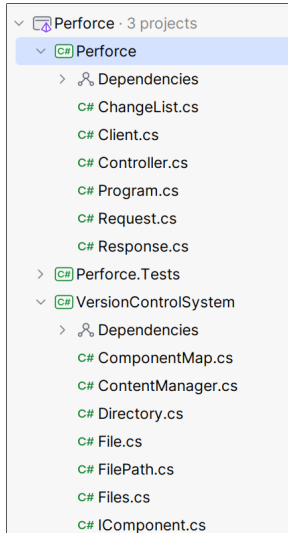
- Si `request.Type` es “client”, se crea un cliente o se obtiene su información.
- Si `request.Type` es “sync”, se obtiene información de los archivo(s) en el depot.
- Si `request.Type` es “submit”, se suben cambios al servidor. Esta solicitud está relacionada con una lista de cambios, los cuales pueden ser agregar, editar o eliminar archivos. Se espera soportar más tipos de cambio en el futuro.

`Handle(...)` retorna un objeto de tipo `Response` con información sobre la ejecución de la `Request` (e.j., si la *request* fue exitosa, la información solicitada, etc).

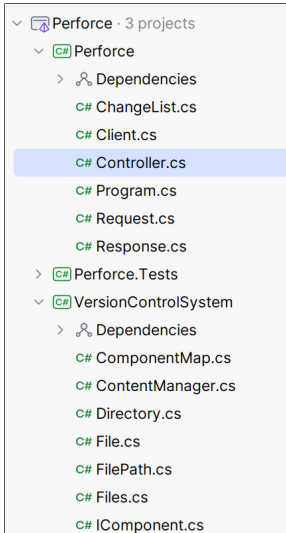
Paso 0: Correr los tests

Paso 1: Entender el Código

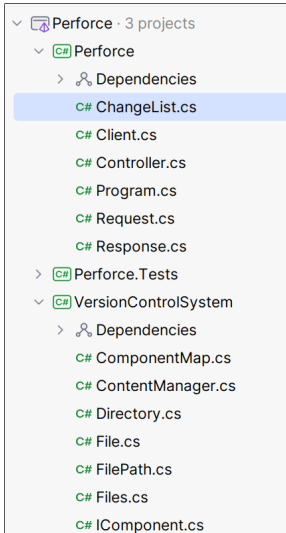
Paso 1: Entender el Código



Paso 1: Entender el Código



Paso 1: Entender el Código



```
1 public class Controller
2 {
3     // Diccionario: nombreUsuario, luego nombreCliente, y finalmente el cliente
4     private Dictionary<string, Dictionary<string, Client>> clients = new ();
5     private IComponent _root;
6
7     public Controller(string path)
8     {
9         // Al inicio, leemos los archivos ya existentes en el servidor desde la
10        carpeta root
11        _root = new VersionControlSystem.Directory();
12        string[] paths = System.IO.Directory.GetFiles(path, "*", SearchOption.
13        AllDirectories);
14        foreach (var path1 in paths)
15        {
16            // Leemos un archivo
17            IEnumerable<string> lines = File.ReadLines(path1);
18            string content = string.Join("\n", lines);
19            // Agregamos el archivo a root. Ojo que debemos eliminar "path" de "
20            path1"
21            // ... FilePath.BuildFromPathAndPrefix(...) hace eso por nosotros
22            _root.Add(FilePath.RemovePrefix(path1, path), content);
23        }
24    }
25 }
```

```

23 public Response Handle(Request request)
24 {
25     Response response = new Response();
26
27     if (request.Type == "client")
28     {
29         if (request.DpotPath == null)
30         {
31             // Cuando el request es "client" pero no se especifica un DepotPath,
32             // se asume que se quiere obtener
33             // la información de ese cliente (por ejemplo, su DepotPath)
34             //
35             // Casos de error: El usuario o cliente no existen
36             if (clients.ContainsKey(request.Username))
37             {
38                 if (clients[request.Username].ContainsKey(request.ClientName))
39                     response.Message = clients[request.Username][request.ClientName].
40 ToString();
41                 else
42                 {
43                     response.IsSuccessfull = false;
44                     response.Message = $"User {request.Username} doesn't have client {
45 request.ClientName}";
46                 }
47             }
48         }
49     }
50 }

```



```
44     }
45     else
46     {
47         response.IsSuccessfull = false;
48         response.Message = $"User {request.Username} not found";
49     }
50 }
51 else
52 {
53     // Cuando el request es "client" y se especifica un DepotPath, se
54     // asume que se quiere crear
55     // un nuevo cliente con el depot entregado
56     //
57     // Casos de error: El cliente ya existe
58     if (_root.Exists(new FilePath(request.DpotPath)))
59     {
60         if (!clients.ContainsKey(request.Username)) clients[request.Username]
61         = new Dictionary<string, Client>();
62         clients[request.Username][request.ClientName] = new Client(request.
63         ClientName, request.DpotPath);
64     }
65     else
66     {
67         response.IsSuccessfull = false;
```

```

65         response.Message = "Invalid depot path";
66     }
67 }
68 } else if ( request.Type == "sync")
69 {
70     if (clients.ContainsKey(request.Username))
71     {
72         if (clients[request.Username].ContainsKey(request.ClientName))
73         {
74             Client client = clients[request.Username][request.ClientName];
75             if (request.FilePath == null)
76             {
77                 // Cuando el request es "sync" pero no es especifica un FilePath
sobre el cual sincronizar
78                 // se asume que el cliente quiere obtener TODOS los archivos de su
depot
79                 //
80                 // Casos de error: El cliente no existe (manejado más arriba)
81                 FilePath path = new FilePath(client.DpotPath);
82                 response.Files = _root.Get(path, "");
83             }
84             else
85             {

```

```
84     else
85     {
86         // Cuando el request es "sync" y se especifica un FilePath sobre
el cual sincronizar
87         // se sincroniza solo el archivo referido en FilePath
88         //
89         // Casos de error: El cliente no existe (manejado más arriba) o el
archivo no existe
90
91         // El FilePath es la ruta al archivo dentro del depot del cliente.
Para obtener la ruta
92         // absoluta del archivo hay que combinar el client.DepotPath con
request.FilePath
93         FilePath path = FilePath.Combine(client.DepotPath, request.
FilePath!);
94         if (!_root.Exists(path))
95         {
96             response.IsSuccessful = false;
97             response.Message = $"Invalid file path {request.FilePath}";
98         }
99         else
100         {
101             response.Files = _root.Get(path, request.FilePath);
102         }
```

```
103     }
104 }
105 else
106 {
107     response.IsSuccessfull = false;
108     response.Message = $"User {request.Username} doesn't have client {
request.ClientName}";
109 }
110 }
111 else
112 {
113     response.IsSuccessfull = false;
114     response.Message = $"User {request.Username} not found";
115 }
116 } else if (request.Type == "submit")
117 {
118     if (clients.ContainsKey(request.Username))
119     {
120         if (clients[request.Username].ContainsKey(request.ClientName))
121         {
122             // Cuando el request es "submit" se realizan todos los cambios
indicados en ChangeList
123             //
124             // Casos de error: El cliente no existe, los cambios son nulos
```

```
125         // o vacíos, y muchos otros (ver ChangeList.cs)
126         Client client = clients[request.Username][request.ClientName];
127         if (request.ChangeList == null || request.ChangeList.IsEmpty())
128         {
129             response.IsSuccessfull = false;
130             response.Message = "Invalid changelist";
131         }
132         else
133         {
134             try
135             {
136                 request.ChangeList.ApplyChanges(_root, client.DepotPath);
137             }
138             catch (Exception e)
139             {
140                 response.IsSuccessfull = false;
141                 response.Message = e.Message;
142             }
143         }
144     }
145     else
146     {
147         response.IsSuccessfull = false;
```

```
148         response.Message = $"User {request.Username} doesn't have client {
request.ClientName}";
149     }
150 }
151 else
152 {
153     response.IsSuccessful = false;
154     response.Message = $"User {request.Username} not found";
155 }
156 }
157 // Se espera agregar más casos a futuro
158
159 return response;
160 }
161 }
```

Paso 1: Entender el Código

Importante: No es necesario entender **todo** el código para limpiar.

Paso 1: Entender el Código

Importante: No es necesario entender **todo** el código para limpiar.

```
1 public class Controller {
2     // Diccionario: nombreUsuario, nombreCliente, cliente
3     private Dictionary<string, Dictionary<string, Client>> clients = new ();
4     private IComponent _root;
5
6     public Controller(string path) { /* ... */ }
7
8     public Response Handle(Request request) {
9         Response response = new Response();
10        if (request.Type == "client") {
11            if (request.DepotPath == null) { /* obtener info de un cliente */ }
12            else { /* crear un nuevo cliente */ }
13        }
14        else if ( request.Type == "sync") { /* obtener archivos */ }
15        else if (request.Type == "submit") { /* modificar archivos */ }
16        return response;
17    }
18 }
```


Paso 1: Entender el Código

Comencemos entendiendo el rol del diccionario:

```
1 // Diccionario: nombreUsuario, nombreCliente, cliente
2 private Dictionary<string, Dictionary<string, Client>> clients = new ();
```

Paso 1: Entender el Código

Comencemos entendiendo el rol del diccionario:

```
1 // Diccionario: nombreUsuario, nombreCliente, cliente
2 private Dictionary<string, Dictionary<string, Client>> clients = new ();
```

Para obtener un Client tendremos que chequear que el usuario y cliente existan.

```
1 if (clients.ContainsKey(request.Username)) {
2     if (clients[request.Username].ContainsKey(request.ClientName)) {
3         Client client = clients[request.Username][request.ClientName];
4         /* ... */
5     }
6     else {
7         response.IsSuccessful = false;
8         response.Message = $"User {request.Username} doesn't have client {request.
9             ClientName}";
10    }
11    else {
12        response.IsSuccessful = false;
13        response.Message = $"User {request.Username} not found";
14    }
```

Paso 1: Entender el Código

Los chequeos de que el usuario/cliente sean válidos están en **todos** lados.

Paso 1: Entender el Código

Los chequeos de que el usuario/cliente sean válidos están en **todos** lados.

```
1 public Response Handle(Request request) {
2     Response response = new Response();
3
4     if (request.Type == "client") {
5         if (request.DepotPath == null) {
6             if (clients.ContainsKey(request.Username)) {
7                 if (clients[request.Username].ContainsKey(request.ClientName))
8                     response.Message = clients[request.Username][request.ClientName].ToString();
9                 else {
10                     response.IsSuccessful = false;
11                     response.Message = $"User {request.Username} doesn't have client {request.ClientName}";
12                 }
13             }
14             else {
15                 response.IsSuccessful = false;
16                 response.Message = $"User {request.Username} not found";
17             }
18         }
19         else { /* ... acá se crean clientes ... */ }
20     }
21     else if ( request.Type == "sync") {
22         if (clients.ContainsKey(request.Username)) {
23             if (clients[request.Username].ContainsKey(request.ClientName)) {
24                 Client client = clients[request.Username][request.ClientName];
25                 /* ... */
26             }
27             else {
```

Paso 1: Entender el Código

Los chequeos de que el usuario/cliente sean válidos están en **todos** lados.

```
28     response.IsSuccessful = false;
29     response.Message = $"User {request.Username} doesn't have client {request.ClientName}";
30 }
31 }
32 else {
33     response.IsSuccessful = false;
34     response.Message = $"User {request.Username} not found";
35 }
36 }
37 else if (request.Type == "submit") {
38     if (clients.ContainsKey(request.Username)) {
39         if (clients[request.Username].ContainsKey(request.ClientName)) {
40             Client client = clients[request.Username][request.ClientName];
41             /* ... */
42         }
43         else {
44             response.IsSuccessful = false;
45             response.Message = $"User {request.Username} doesn't have client {request.ClientName}";
46         }
47     }
48     else {
49         response.IsSuccessful = false;
50         response.Message = $"User {request.Username} not found";
51     }
52 }
53 return response;
54 }
```

Paso 1: Entender el Código

Arreglar esto requiere combinar cosas del cap. 8 y del cap. 7.

Paso 1: Entender el Código

Arreglar esto requiere combinar cosas del cap. 8 y del cap. 7.

1) Hay que encapsular este diccionario:

```
1 // Diccionario: nombreUsuario, nombreCliente, cliente
2 private Dictionary<string, Dictionary<string, Client>> clients = new ();
```

Paso 1: Entender el Código

Arreglar esto requiere combinar cosas del cap. 8 y del cap. 7.

1) Hay que encapsular este diccionario:

```
1 // Diccionario: nombreUsuario, nombreCliente, cliente
2 private Dictionary<string, Dictionary<string, Client>> clients = new ();
```

2) Al momento de pedir un Client, tirar una excepción si el usuario o cliente no existe.

Paso 1: Entender el Código

Arreglar esto requiere combinar cosas del cap. 8 y del cap. 7.

1) Hay que encapsular este diccionario:

```
1 // Diccionario: nombreUsuario, nombreCliente, cliente  
2 private Dictionary<string, Dictionary<string, Client>> clients = new ();
```

2) Al momento de pedir un Client, tirar una excepción si el usuario o cliente no existe.

3) Borrar todos los ifs que chequean que los usuarios/clientes son correctos del Handle.

Paso 1: Entender el Código

Arreglar esto requiere combinar cosas del cap. 8 y del cap. 7.

1) Hay que encapsular este diccionario:

```
1 // Diccionario: nombreUsuario, nombreCliente, cliente
2 private Dictionary<string, Dictionary<string, Client>> clients = new ();
```

2) Al momento de pedir un Client, tirar una excepción si el usuario o cliente no existe.

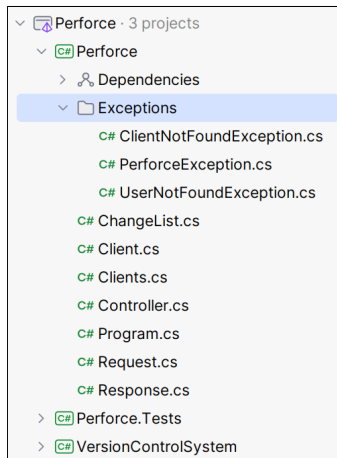
3) Borrar todos los ifs que chequean que los usuarios/clientes son correctos del Handle.

4) Atajar las excepción en Handle y retornar una invalid request en ese caso.

Boundaries 🤝 Exceptions

Boundaries & Exceptions

Comencemos creando las excepciones que necesitamos (con sus mensajes de error).



Boundaries & Exceptions

Comencemos creando las excepciones que necesitamos (con sus mensajes de error).

```
1 abstract class PerforceException : ApplicationException {  
2     public abstract string GetErrorMessage();  
3 }
```

Boundaries & Exceptions

Comencemos creando las excepciones que necesitamos (con sus mensajes de error).

```
1 abstract class PerforceException : ApplicationException {  
2     public abstract string GetErrorMessage();  
3 }
```

Luego tenemos una excepción para usuario inválido.

```
1 class UserNotFoundException(string userName) : PerforceException {  
2     public override string GetErrorMessage()  
3     => $"User {userName} not found";  
4 }
```

Boundaries & Exceptions

Comencemos creando las excepciones que necesitamos (con sus mensajes de error).

```
1 abstract class PerforceException : ApplicationException {  
2     public abstract string GetErrorMessage();  
3 }
```

Luego tenemos una excepción para usuario inválido.

```
1 class UserNotFoundException(string userName) : PerforceException {  
2     public override string GetErrorMessage()  
3     => $"User {userName} not found";  
4 }
```

... y otra para cliente inválido.

```
1 class ClientNotFoundException(string userName, string clientName) :  
2     PerforceException {  
3     public override string GetErrorMessage()  
4     => $"User {userName} doesn't have client {clientName}";  
5 }
```

Boundaries & Exceptions

Ahora encapsulamos el diccionario y tiramos nuestras excepciones:

```
1 public class Clients {
2     private Dictionary<string, Dictionary<string, Client>> _clients = new ();
3
4     public void Add(Request request) {
5         if (!_clients.ContainsKey(request.Username))
6             _clients[request.Username] = new Dictionary<string, Client>();
7         _clients[request.Username][request.ClientName] = new Client(request.
            ClientName, request.DepotPath);
8     }
9
10    public Client Get(Request request) {
11        if (!_clients.ContainsKey(request.Username))
12            throw new UserNotFoundException(request.Username);
13        if (!_clients[request.Username].ContainsKey(request.ClientName))
14            throw new ClientNotFoundException(request.Username, request.ClientName);
15        return _clients[request.Username][request.ClientName];
16    }
17 }
```


Boundaries & Exceptions

Recordemos que el método `Handle` se veía así:

Boundaries & Exceptions

Recordemos que el método Handle se veía así:

```
1 public Response Handle(Request request) {
2     Response response = new Response();
3
4     if (request.Type == "client") {
5         if (request.DepotPath == null) {
6             if (clients.ContainsKey(request.Username)) {
7                 if (clients[request.Username].ContainsKey(request.ClientName))
8                     response.Message = clients[request.Username][request.ClientName].ToString();
9                 else {
10                     response.IsSuccessful = false;
11                     response.Message = $"User {request.Username} doesn't have client {request.ClientName}";
12                 }
13             }
14             else {
15                 response.IsSuccessful = false;
16                 response.Message = $"User {request.Username} not found";
17             }
18         }
19         else { /* ... acá se crean clientes ... */ }
20     }
21     else if ( request.Type == "sync") {
22         if (clients.ContainsKey(request.Username)) {
23             if (clients[request.Username].ContainsKey(request.ClientName)) {
24                 Client client = clients[request.Username][request.ClientName];
25                 /* ... */
26             }
27             else {
```

Boundaries & Exceptions

Recordemos que el método Handle se veía así:

```
28     response.IsSuccessful = false;
29     response.Message = $"User {request.Username} doesn't have client {request.ClientName}";
30 }
31 }
32 else {
33     response.IsSuccessful = false;
34     response.Message = $"User {request.Username} not found";
35 }
36 }
37 else if (request.Type == "submit") {
38     if (clients.ContainsKey(request.Username)) {
39         if (clients[request.Username].ContainsKey(request.ClientName)) {
40             Client client = clients[request.Username][request.ClientName];
41             /* ... */
42         }
43         else {
44             response.IsSuccessful = false;
45             response.Message = $"User {request.Username} doesn't have client {request.ClientName}";
46         }
47     }
48     else {
49         response.IsSuccessful = false;
50         response.Message = $"User {request.Username} not found";
51     }
52 }
53 return response;
54 }
```

Boundaries & Exceptions

Gracias a Clients, las 54 líneas anteriores se reducen a esto:

```
1 private Response TryToHandle(Request request) {  
2     Response response = new Response();  
3  
4     if (request.Type == "client") {  
5         if (request.DepotPath == null)  
6             response.Message = _clients.Get(request).ToString();  
7         else { /* ... acá se crean clientes ... */ }  
8     }  
9     else if ( request.Type == "sync") {  
10         Client client = _clients.Get(request);  
11         /* ... */  
12     }  
13     else if (request.Type == "submit") {  
14         Client client = _clients.Get(request);  
15         /* ... */  
16     }  
17  
18     return response;  
19 }
```

Boundaries & Exceptions

Sin embargo, necesitamos atajar la excepción y agregar clientes.

Boundaries & Exceptions

Sin embargo, necesitamos atajar la excepción y agregar clientes.

```
1 public class Controller {
2     private Clients _clients = new ();
3     /* ... */
4
5     public Response Handle(Request request) {
6         try {
7             return TryToHandle(request);
8         }
9         catch (PerforceException e) {
10             string errorMessage = e.GetErrorMessage();
11             return Response.BuildUnsuccessfulResponse(errorMessage);
12         }
13     }
14
15     private Response TryToHandle(Request request) {
16         Response response = new Response();
17
18         if (request.Type == "client") {
19             if (request.DepotPath == null)
20                 response.Message = _clients.Get(request).ToString();
21             else {
```

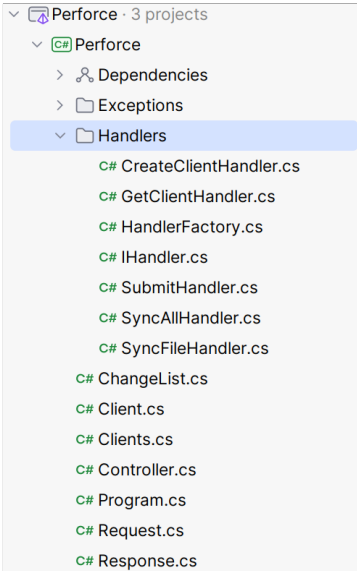
Boundaries & Exceptions

Sin embargo, necesitamos atajar la excepción y agregar clientes.

```
22         if (_root.Exists(new FilePath(request.DepotPath)))
23             _clients.Add(request);
24         else {
25             response.IsSuccessful = false;
26             response.Message = "Invalid depot path";
27         }
28     }
29 }
30 else if (request.Type == "sync") {
31     Client client = _clients.Get(request);
32     /* ... */
33 }
34 else if (request.Type == "submit") {
35     Client client = _clients.Get(request);
36     /* ... */
37 }
38
39 return response;
40 }
41 }
```

Objects vs Data Structures


```
1 private Response TryToHandle(Request request) {
2     Response response = new Response();
3     if (request.Type == "client") {
4         if (request.DepotPath == null) {
5             /* Obtener cliente */
6         }
7         else {
8             /* Crear cliente */
9         }
10    }
11    else if (request.Type == "sync") {
12        if (request.FilePath == null) {
13            /* Obtener todos los archivos */
14        }
15        else {
16            /* Obtener un archivo */
17        }
18    }
19    else if (request.Type == "submit") {
20        /* Modificar archivos existentes */
21    }
22    return response;
23 }
```



Objects vs Data Structures

No nos compliquemos y hagamos esto de la forma más trivial posible.

```
1 public interface IHandler {  
2     Response Handle(Request request);  
3 }
```

Objects vs Data Structures

No nos compliquemos y hagamos esto de la forma más trivial posible.

```
1 public interface IHandler {  
2     Response Handle(Request request);  
3 }
```

Luego tenemos una factory para crear el handler apropiado:

```
1 public class HandlerFactory(Clients clients, IComponent root) {  
2     public IHandler Create(Request request) {  
3         if (request.Type == "client" && request.DepotPath == null)  
4             return new GetClientHandler(clients);  
5         if (request.Type == "client" && request.DepotPath != null)  
6             return new CreateClientHandler(clients, root);  
7         if (request.Type == "sync" && request.FilePath == null)  
8             return new SyncAllHandler(clients, root);  
9         if (request.Type == "sync" && request.FilePath != null)  
10            return new SyncFileHandler(clients, root);  
11        if (request.Type == "submit")  
12            return new SubmitHandler(clients, root);  
13        throw new NotImplementedException();  
14    }  
15 }
```

Objects vs Data Structures

Con esto el Controller queda casi listo:

```
1 public class Controller {
2     private HandlerFactory _factory;
3     /* ... */
4     public Response Handle(Request request) {
5         try {
6             return TryToHandle(request);
7         }
8         catch (PerforceException e) {
9             string errorMessage = e.GetErrorMessage();
10            return Response.BuildUnsuccessfulResponse(errorMessage);
11        }
12    }
13
14    private Response TryToHandle(Request request) {
15        IHandler handler = _factory.Create(request);
16        return handler.Handle(request);
17    }
18 }
```

Objects vs Data Structures

El código de cada Handler, por ahora, es el mismo código que estaba en los ifs.

```
1 class CreateClientHandler(Clients clients, IComponent root) : IHandler {
2     public Response Handle(Request request) {
3         Response response = new Response();
4
5         if (root.Exists(new FilePath(request.DepotPath)))
6             clients.Add(request);
7         else {
8             response.IsSuccessful = false;
9             response.Message = "Invalid depot path";
10        }
11
12        return response;
13    }
14 }
```

Objects vs Data Structures

El código de cada Handler, por ahora, es el mismo código que estaba en los ifs.

```
1 class CreateClientHandler(Clients clients, IComponent root) : IHandler {  
2     public Response Handle(Request request) {  
3         Response response = new Response();  
4  
5         if (root.Exists(new FilePath(request.DepotPath)))  
6             clients.Add(request);  
7         else {  
8             response.IsSuccessful = false;  
9             response.Message = "Invalid depot path";  
10        }  
11  
12        return response;  
13    }  
14 }
```

... aprovechemos de limpiar un poco esta clase.

Objects vs Data Structures

Creamos una excepción para el invalid depot:

```
1 public class InvalidDepotException : PerforceException {  
2     public override string GetErrorMessage()  
3         => "Invalid depot path";  
4 }
```


Objects vs Data Structures

Creamos una excepción para el invalid depot:

```
1 public class InvalidDepotException : PerforceException {  
2     public override string GetErrorMessage()  
3         => "Invalid depot path";  
4 }
```

Ahora simplemente lanzamos la excepción si el depot es inválido:

```
1 class CreateClientHandler(Clients clients, IComponent root) : IHandler {  
2     public Response Handle(Request request) {  
3         if (!root.Exists(new FilePath(request.DepotPath)))  
4             throw new InvalidDepotException();  
5         Response response = new Response();  
6         clients.Add(request);  
7         return response;  
8     }  
9 }
```

Objects vs Data Structures

El Handler para obtener la información del usuario ya está como listo:

```
1 class GetClientHandler(Clients clients) : IHandler {  
2     public Response Handle(Request request) {  
3         Response response = new Response();  
4         Client client = clients.Get(request);  
5         response.Message = client.ToString();  
6         return response;  
7     }  
8 }
```

Objects vs Data Structures

El Handler para obtener la información del usuario ya está como listo:

```
1 class GetClientHandler(Clients clients) : IHandler {
2     public Response Handle(Request request) {
3         Response response = new Response();
4         Client client = clients.Get(request);
5         response.Message = client.ToString();
6         return response;
7     }
8 }
```

El Handler para obtener todos los archivos igual ya está bastante bien:

```
1 class SyncAllHandler(Clients clients, IComponent root) : IHandler {
2     public Response Handle(Request request) {
3         Response response = new Response();
4         Client client = clients.Get(request);
5         FilePath path = new FilePath(client.DepotPath);
6         response.Files = root.Get(path, "");
7         return response;
8     }
9 }
```

Objects vs Data Structures

Algo se puede limpiar el Handler para obtener un archivo:

```
1 class SyncFileHandler(Clients clients, IComponent root) : IHandler {
2     public Response Handle(Request request) {
3         Response response = new Response();
4
5         Client client = clients.Get(request);
6         FilePath path = FilePath.Combine(client.DepotPath, request.FilePath!);
7         if (!root.Exists(path)) {
8             response.IsSuccessful = false;
9             response.Message = $"Invalid file path {request.FilePath}";
10        }
11        else
12            response.Files = root.Get(path, request.FilePath);
13
14        return response;
15    }
16 }
```

Objects vs Data Structures

Una vez más, creamos una excepción:

```
1 public class InvalidPathException(string filePath) : PerforceException {  
2     public override string GetErrorMessage()  
3         => $"Invalid file path {filePath}";  
4 }
```

Objects vs Data Structures

Una vez más, creamos una excepción:

```
1 public class InvalidPathException(string filePath) : PerforceException {  
2     public override string GetErrorMessage()  
3         => $"Invalid file path {filePath}";  
4 }
```

Lanzamos la excepción:

```
1 class SyncFileHandler(Clients clients, IComponent root) : IHandler {  
2     public Response Handle(Request request) {  
3         Client client = clients.Get(request);  
4         FilePath path = FilePath.Combine(client.DepotPath, request.FilePath!);  
5         if (!root.Exists(path))  
6             throw new InvalidPathException(request.FilePath);  
7         Response response = new Response();  
8         response.Files = root.Get(path, request.FilePath);  
9         return response;  
10    }  
11 }
```

Objects vs Data Structures

Ya, y el Handler que de verdad requiere trabajo es el para modificar archivos:

```
1  class SubmitHandler(Clients clients, IComponent root) : IHandler {
2      public Response Handle(Request request) {
3          Response response = new Response();
4          Client client = clients.Get(request);
5          if (request.ChangeList == null || request.ChangeList.IsEmpty()) {
6              response.IsSuccessful = false;
7              response.Message = "Invalid changelist";
8          }
9          else {
10             try {
11                 request.ChangeList.ApplyChanges(root, client.DepotPath);
12             }
13             catch (Exception e) {
14                 response.IsSuccessful = false;
15                 response.Message = e.Message;
16             }
17         }
18         return response;
19     }
20 }
```

Objects vs Data Structures

Partamos por lo obvio:

```
1 class SubmitHandler(Clients clients, IComponent root) : IHandler {
2     public Response Handle(Request request) {
3         Client client = clients.Get(request);
4         if (request.ChangeList == null || request.ChangeList.IsEmpty())
5             throw new InvalidChangeListException();
6
7         Response response = new Response();
8         try {
9             request.ChangeList.ApplyChanges(root, client.DepotPath);
10        }
11        catch (Exception e) {
12            response.IsSuccessful = false;
13            response.Message = e.Message;
14        }
15        return response;
16    }
17 }
```


Objects vs Data Structures

Partamos por lo obvio:

```
1 class SubmitHandler(Clients clients, IComponent root) : IHandler {
2     public Response Handle(Request request) {
3         Client client = clients.Get(request);
4         if (request.ChangeList == null || request.ChangeList.IsEmpty())
5             throw new InvalidChangeListException();
6
7         Response response = new Response();
8         try {
9             request.ChangeList.ApplyChanges(root, client.DepotPath);
10        }
11        catch (Exception e) {
12            response.IsSuccessful = false;
13            response.Message = e.Message;
14        }
15        return response;
16    }
17 }
```

... para limpiar ese try-catch tenemos que meternos al código del ChangeList.

Paso 1: Entender el Código

```
1 public class ChangeList {
2     // (Tipo de cambio, ruta al archivo a cambiar, nuevo contenido del archivo)
3     // si el contenido es null significa que el archivo debe ser borrado
4     private List<(string, string, string?)> Changes = new();
5
6     public ChangeList(string type, string[] filePaths) {
7         for (int i = 0; i < filePaths.Length; i++)
8             Changes.Add((type, filePaths[i], null));
9     }
10
11     public ChangeList(string type, string[] filePaths, string[] contents) {
12         for (int i = 0; i < filePaths.Length; i++)
13             Changes.Add((type, filePaths[i], contents[i]));
14     }
15
16     public ChangeList() {
17
18     }
19
20     public object GetPathByIndex(int index)
21         => Changes[index].Item2;
22
23     public bool IsEmpty()
```

```
24 => Changes.Count == 0;
25
26 public void Validate(IComponent _root, string path1) {
27     foreach (var change in Changes) {
28         FilePath path = FilePath.Combine(path1, change.Item2);
29         if (change.Item1 == "add") {
30             // Add significa agregar un nuevo archivo.
31             // Esto falla si es que el archivo ya existe
32             // o si el contenido del archivo que se quiere agregar es "null"
33             if (_root.Exists(path))
34                 throw new Exception($"Invalid add change: File {change.Item2}
already exists");
35
36             if (change.Item3 == null)
37                 throw new Exception($"Invalid add change: File {change.Item2} has no
content");
38         }
39         if (change.Item1 == "delete") {
40             // Para borrar un archivo se tiene que ingresar la ruta del archivo
41             // a borrar y decir que su nuevo contenido es "null". Por eso la
42             // operación falla si es que el archivo no existe o el contenido
43             // a subir es distinto de null
44             if (!_root.Exists(path))
45                 throw new Exception($"Invalid delete change: File {change.Item2}
doesn't exists");
```

```
46         if (change.Item3 != null)
47             throw new Exception($"Invalid delete change: File {change.Item2} has
48 content");
49     }
50     if (change.Item1 == "edit") {
51         // Para editar un archivo, el archivo debe existir y el nuevo
52 contenido
53         // NO puede ser null... de lo contrario, equivaldría a borrar dicho
54 archivo.
55         if (!_root.Exists(path))
56             throw new Exception($"Invalid edit change: File {change.Item2} doesn
57 't exists");
58
59         if (change.Item3 == null)
60             throw new Exception($"Invalid edit change: File {change.Item2} has
61 no content");
62     }
63     // Se espera agregar más casos a futuro
64 }
65
66 public void ApplyChanges(IComponent root, string path1) {
67     Validate(root, path1); // acá path1 es el depot
```

```
65     foreach (var change in Changes) {
66         // change.Item2 es la ruta del archivo desde el depot del cliente
67         // por eso para obtener la ruta absoluta al archivo hay que combinar
68         // la ruta del depot con change.Item2
69         FilePath path = FilePath.Combine(path1, change.Item2);
70         root.Add(path, change.Item3);
71     }
72 }
73 }
```

Paso 1: Entender el Código

Cosas a destacar. Hay una tupla que debería ser una clase:

```
1 // (Tipo de cambio, ruta al archivo a cambiar, nuevo contenido del archivo)
2 private List<(string, string, string?)> Changes = new();
```

Paso 1: Entender el Código

Cosas a destacar. Hay una tupla que debería ser una clase:

```
1 // (Tipo de cambio, ruta al archivo a cambiar, nuevo contenido del archivo)
2 private List<(string, string, string?)> Changes = new();
```

Hay excepciones genéricas que deberían ser PerforceExceptions:

```
1 if (!_root.Exists(path))
2     throw new Exception("Invalid add change: File already exists");
```


Paso 1: Entender el Código

Cosas a destacar. Hay una tupla que debería ser una clase:

```
1 // (Tipo de cambio, ruta al archivo a cambiar, nuevo contenido del archivo)
2 private List<(string, string, string?)> Changes = new();
```

Hay excepciones genéricas que deberían ser PerforceExceptions:

```
1 if (!_root.Exists(path))
2     throw new Exception("Invalid add change: File already exists");
```

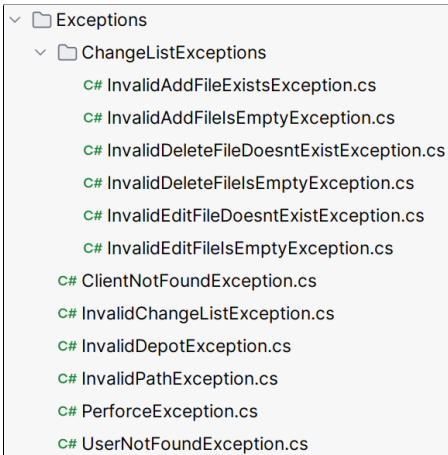
Hay una cadena de if que seguirá creciendo:

```
1 FilePath path = FilePath.Combine(path1, change.Item2);
2 if (change.Item1 == "add") {
3     /* ... */
4 }
5 if (change.Item1 == "delete") {
6     /* ... */
7 }
8 if (change.Item1 == "edit") {
9     /* ... */
10 } // Se espera agregar más casos a futuro
```

Exceptions

Exceptions

Acá no hay ninguna magia... simplemente creamos las excepciones nuevas:



Exceptions

Acá no hay ninguna magia... simplemente creamos las excepciones nuevas:

```
1 class InvalidAddFileExistsException(string file) : PerforceException {  
2     public override string GetErrorMessage()  
3     => $"Invalid add change: File {file} already exists";  
4 }
```

Exceptions

Acá no hay ninguna magia... simplemente creamos las excepciones nuevas:

```
1  FilePath path = FilePath.Combine(path1, change.Item2);
2  if (change.Item1 == "add") {
3      if (!_root.Exists(path))
4          throw new InvalidAddFileExistsException(change.Item2);
5      if (change.Item3 == null)
6          throw new InvalidAddFileIsEmptyException(change.Item2);
7  }
8  if (change.Item1 == "delete") {
9      if (!_root.Exists(path))
10         throw new InvalidDeleteFileDoesntExistException(change.Item2);
11     if (change.Item3 != null)
12         throw new InvalidDeleteFileIsEmptyException(change.Item2);
13 }
14 if (change.Item1 == "edit") {
15     if (!_root.Exists(path))
16         throw new InvalidEditFileDoesntExistException(change.Item2);
17     if (change.Item3 == null)
18         throw new InvalidEditFileIsEmptyException(change.Item2);
19 }
```

Exceptions

Acá no hay ninguna magia... simplemente creamos las excepciones nuevas:

```
1 class SubmitHandler(Clients clients, IComponent root) : IHandler {
2     public Response Handle(Request request) {
3         Client client = clients.Get(request);
4         if (request.ChangeList == null || request.ChangeList.IsEmpty())
5             throw new InvalidChangeListException();
6
7         Response response = new Response();
8         try {
9             request.ChangeList.ApplyChanges(root, client.DepotPath);
10        }
11        catch (Exception e) {
12            response.IsSuccessful = false;
13            response.Message = e.Message;
14        }
15        return response;
16    }
17 }
```

Exceptions

Acá no hay ninguna magia... simplemente creamos las excepciones nuevas:

```
1 class SubmitHandler(Clients clients, IComponent root) : IHandler {
2     public Response Handle(Request request) {
3         Client client = clients.Get(request);
4         if (request.ChangeList == null || request.ChangeList.IsEmpty())
5             throw new InvalidChangeListException();
6
7         Response response = new Response();
8         request.ChangeList.ApplyChanges(root, client.DepotPath);
9         return response;
10    }
11 }
```

Classes >> Tuples

Transformemos la tupla en una clase:

```
2 private List<(string, string, string?)> Changes = new();
```

Classes >> Tuplas

Transformemos la tupla en una clase:

```
2 private List<(string, string, string?)> Changes = new();
```

Primero creo una clase que reemplace la tupla:

```
1 class Change(string changeType, string path, string? content) {  
2     public string ChangeType { get; } = changeType;  
3     public string Path { get; } = path;  
4     public string? Content { get; } = content;  
5 }
```

Clases >> Tuplas

Transformemos la tupla en una clase:

```
2 private List<(string, string, string?)> Changes = new();
```

Primero creo una clase que reemplace la tupla:

```
1 class Change(string changeType, string path, string? content) {  
2     public string ChangeType { get; } = changeType;  
3     public string Path { get; } = path;  
4     public string? Content { get; } = content;  
5 }
```

Ahora simplemente usamos la clase en vez de la tupla:

```
1 public class ChangeList {  
2     private readonly List<Change> _changes = new();  
3  
4     public ChangeList(string type, string[] filePaths) {  
5         for (int i = 0; i < filePaths.Length; i++)  
6             _changes.Add(new Change(type, filePaths[i], null));  
7     }  
8     /* ... */  
9 }
```

Objects vs Data Structures

Objects vs Data Structures

Hay una cadena de if que seguirá creciendo:

```
1 FilePath path = FilePath.Combine(path1, change.Item2);
2 if (change.Item1 == "add") {
3     /* ... */
4 }
5 if (change.Item1 == "delete") {
6     /* ... */
7 }
8 if (change.Item1 == "edit") {
9     /* ... */
10 } // Se espera agregar más casos a futuro
```

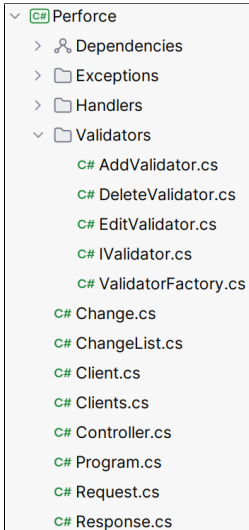
Objects vs Data Structures

Hay una cadena de if que seguirá creciendo:

```
1 FilePath path = FilePath.Combine(path1, change.Item2);
2 if (change.Item1 == "add") {
3     /* ... */
4 }
5 if (change.Item1 == "delete") {
6     /* ... */
7 }
8 if (change.Item1 == "edit") {
9     /* ... */
10 } // Se espera agregar más casos a futuro
```

Solución: Que sea una herencia con un factory (igual que antes).

Objects vs Data Structures



Objects vs Data Structures

La interfaz de más o menos lo que uno esperaría:

```
1 interface IValidator {  
2     void Validate(Change change);  
3 }
```


Objects vs Data Structures

La interfaz de más o menos lo que uno esperaría:

```
1 interface IValidator {  
2     void Validate(Change change);  
3 }
```

Cada validador funciona como esperas:

```
1 class AddValidator(IComponent root, string depot) : IValidator {  
2     public void Validate(Change change) {  
3         FilePath path = FilePath.Combine(depot, change.Path);  
4         if (root.Exists(path))  
5             throw new InvalidAddFileExistsException(change.Path);  
6         if (change.Content == null)  
7             throw new InvalidAddFileIsEmptyException(change.Path);  
8     }  
9 }
```

Objects vs Data Structures

Luego solo falta nuestra factory:

```
1 class ValidatorFactory {  
2     public IValidator Create(IComponent root, string depot, string changeType) {  
3         if (changeType == "add")  
4             return new AddValidator(root, depot);  
5         if (changeType == "delete")  
6             return new DeleteValidator(root, depot);  
7         if (changeType == "edit")  
8             return new EditValidator(root, depot);  
9         throw new NotImplementedException();  
10    }  
11 }
```

... y cambiar el método del ChangeList:

```
1 public class ChangeList {
2     private readonly ValidatorFactory _factory = new();
3     private readonly List<Change> _changes = new();
4
5     public ChangeList(string type, string[] filePaths) {
6         for (int i = 0; i < filePaths.Length; i++)
7             _changes.Add(new Change(type, filePaths[i], null));
8     }
9
10    public ChangeList(string type, string[] filePaths, string[] contents) {
11        for (int i = 0; i < filePaths.Length; i++)
12            _changes.Add(new Change(type, filePaths[i], contents[i]));
13    }
14
15    public ChangeList() {
16
17    }
18
19    public object GetPathByIndex(int index)
20        => _changes[index].Path;
21
22    public bool IsEmpty()
23        => _changes.Count == 0;
```

```
25 public void ApplyChanges(IComponent root, string depot) {
26     Validate(root, depot);
27     foreach (var change in _changes) {
28         FilePath path = FilePath.Combine(depot, change.Path);
29         root.Add(path, change.Content);
30     }
31 }
32
33 private void Validate(IComponent root, string depot) {
34     foreach (var change in _changes) {
35         IValidator validator = _factory.Create(root, depot, change.ChangeType);
36         validator.Validate(change);
37     }
38 }
39 }
```

**Ahora solo faltan maquillajes
menores al código**

Maquillajes menores

```
1 public class Controller {
2     private readonly HandlerFactory _factory;
3     private readonly Clients _clients = new ();
4
5     public Controller(string path) {
6         // Al inicio, leemos los archivos ya existentes en el servidor desde la
6         // carpeta root
7         IComponent root = new VersionControlSystem.Directory();
8         string[] paths = System.IO.Directory.GetFiles(path, "*", SearchOption.
8         AllDirectories);
9         foreach (var path1 in paths) {
10             // Leemos un archivo
11             IEnumerable<string> lines = File.ReadLines(path1);
12             string content = string.Join("\n", lines);
13             // Agregamos el archivo a root. Ojo que debemos eliminar "path" de "
13             path1"
14             // ... FilePath.BuildFromPathAndPrefix(...) hace eso por nosotros
15             root.Add(FilePath.RemovePrefix(path1, path), content);
16         }
17
18         _factory = new HandlerFactory(_clients, root);
19     }
```

Maquillajes menores

```
21 public Response Handle(Request request) {
22     try {
23         return TryToHandle(request);
24     }
25     catch (PerforceException e) {
26         string errorMessage = e.GetErrorMessage();
27         return Response.BuildUnsuccessfulResponse(errorMessage);
28     }
29 }
30
31 private Response TryToHandle(Request request) {
32     IHandler handler = _factory.Create(request);
33     return handler.Handle(request);
34 }
35 }
```

Maquillajes menores

```
21 public Response Handle(Request request) {  
22     try {  
23         return TryToHandle(request);  
24     }  
25     catch (PerforceException e) {  
26         string errorMessage = e.GetErrorMessage();  
27         return Response.BuildUnsuccessfulResponse(errorMessage);  
28     }  
29 }  
30  
31 private Response TryToHandle(Request request) {  
32     IHandler handler = _factory.Create(request);  
33     return handler.Handle(request);  
34 }  
35 }
```

Cosas feas: El constructor y que `IComponent` es una interfaz que no depende de nosotros (i.e., deberíamos encapsular).


```
1 public class VersionSystem {
2     private IComponent _root;
3
4     public VersionSystem(string rootPath) {
5         _root = new VersionControlSystem.Directory();
6         string[] paths = System.IO.Directory.GetFiles(rootPath, "*", SearchOption.
7             AllDirectories);
8         foreach (var path in paths)
9             AddFileToRoot(rootPath, path);
10    }
11
12    private void AddFileToRoot(string rootPath, string path) {
13        IEnumerable<string> lines = File.ReadLines(path);
14        string content = string.Join("\n", lines);
15        _root.Add(FilePath.RemovePrefix(path, rootPath), content);
16    }
17
18    public void Add(FilePath filePath, string? content)
19        => _root.Add(filePath, content);
20
21    public bool Exists(FilePath path)
22        => _root.Exists(path);
23
24    public Files Get(FilePath path, string pathToFile)
25        => _root.Get(path, pathToFile);
26    }
```

```
1 public class Controller {
2     private readonly HandlerFactory _factory;
3     private readonly Clients _clients = new ();
4
5     public Controller(string rootPath) {
6         VersionSystem versionSystem = new VersionSystem(rootPath);
7         _factory = new HandlerFactory(_clients, versionSystem);
8     }
9
10    public Response Handle(Request request) {
11        try {
12            return TryToHandle(request);
13        }
14        catch (PerforceException e) {
15            string errorMessage = e.GetErrorMessage();
16            return Response.BuildUnsuccessfulResponse(errorMessage);
17        }
18    }
19
20    private Response TryToHandle(Request request) {
21        IHandler handler = _factory.Create(request);
22        return handler.Handle(request);
23    }
24 }
```

**¡Listo! Ya limpiamos
todo el código**